

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA TOÁN - CƠ - TIN HỌC**



BÁO CÁO KẾT THÚC THỰC TẬP

**Ứng dụng WebSocket trong việc xây dựng trang web xem
video đồng bộ trạng thái độ trễ thấp**

Họ và tên sinh viên: Tạ Tiến Nam

Mã sinh viên: 23000143

Lớp: K68A2 Toán Tin

Ngành: Toán Tin

Khoa: Toán - Cơ - Tin học

Trường: Đại học Khoa học Tự nhiên (VNU - HUS)

Ngày sinh: 22/07/2005

Email: namxivin@gmail.com

Hà Nội, Năm 2026

LỜI CẢM ƠN

Lời đầu tiên, em xin gửi lời cảm ơn chân thành và sâu sắc nhất tới toàn thể các thầy cô giáo trong Khoa Toán - Cơ - Tin học, Trường Đại học Khoa học Tự nhiên – Đại học Quốc gia Hà Nội đã tận tình giảng dạy, truyền đạt những kiến thức chuyên môn vững chắc và tư duy logic quý báu cho em trong suốt quá trình học tập tại trường.

Đặc biệt, em xin bày tỏ lòng biết ơn sâu sắc tới các thầy cô hướng dẫn môn học Thực tập chuyên ngành đã luôn theo sát, định hướng chuyên môn quý báu giúp em hoàn thiện đề tài **”Ứng dụng WebSocket trong việc xây dựng trang web xem video đồng bộ trạng thái độ trễ thấp”**.

Dù đã cố gắng hoàn thiện báo cáo và ứng dụng một cách cẩn chu, bài báo cáo không tránh khỏi những hạn chế nhất định. Em rất mong nhận được sự đóng góp ý kiến từ phía thầy cô để tiếp tục phát triển sản phẩm tốt hơn.

Em xin chân thành cảm ơn!

Hà Nội, ngày 15 tháng 08 năm 2026

Sinh viên thực hiện

Tạ Tiến Nam

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tên tiếng Anh đầy đủ	Ý nghĩa tiếng Việt
API	Application Programming Interface	Giao diện lập trình ứng dụng
CORS	Cross-Origin Resource Sharing	Chia sẻ tài nguyên khác nguồn
HTTP / HTTPS	Hypertext Transfer Protocol (Secure)	Giao thức truyền tải siêu văn bản (Bảo mật)
JSON	JavaScript Object Notation	Định dạng trao đổi dữ liệu JSON
JWT	JSON Web Token	Mã thông báo xác thực chuỗi mã hóa
REST	Representational State Transfer	Kiểu kiến trúc truyền tải trạng thái đại diện
SPA	Single Page Application	Ứng dụng trang đơn
WebSocket	WebSocket Protocol	Giao thức truyền thông 2 chiều qua TCP
RDBMS	Relational Database Management System	Hệ quản trị cơ sở dữ liệu quan hệ

Danh sách bảng

2	Cấu trúc các bảng cơ sở dữ liệu PostgreSQL của hệ thống	9
3	Bảng tổng hợp kết quả kiểm thử các chức năng chính	11
4	Đánh giá độ trễ đồng bộ video thời gian thực trong các kịch bản mạng	12

Danh sách hình vẽ

1	Sơ đồ luồng xử lý đồng bộ trạng thái phát video thời gian thực	9
---	--	---

Mục lục

LỜI CẢM ƠN	1
DANH MỤC TỪ VIẾT TẮT	2
DANH MỤC BẢNG BIỂU	3
DANH MỤC HÌNH ẢNH, SƠ ĐỒ	4
1 PHẦN 1: GIỚI THIỆU	7
1.1 Phát biểu bài toán và công việc được giao	7
1.2 Khảo sát công nghệ, công cụ và lý do lựa chọn	7
1.2.1 Truyền thông thời gian thực: HTTP Polling vs WebSocket	7
1.2.2 Xác thực và Bảo mật: Session-Cookie vs JWT + Bcrypt	7
1.2.3 Backend & Cơ sở dữ liệu: Node.js, PostgreSQL và Redis	8
2 PHẦN 2: CÔNG VIỆC TRIỂN KHAI	9
2.1 Phân tích yêu cầu và Thiết kế kiến trúc hệ thống	9
2.1.1 Thiết kế Cơ sở dữ liệu PostgreSQL	9
2.1.2 Sơ đồ luồng dữ liệu	9
2.2 Triển khai các Module chức năng	10
2.2.1 Module Xác thực và Quản lý phòng xem	10
2.2.2 Module Đồng bộ Video và Thuật toán Bù sai lệch	10
2.2.3 Module Trò chuyện nhóm thời gian thực	10
3 PHẦN 3: CÁC KẾT QUẢ ĐẠT ĐƯỢC	11
3.1 Giới thiệu sản phẩm phần mềm và Giao diện	11
3.2 Kết quả kiểm thử và Phân tích lỗi kỹ thuật	11
3.2.1 Kết quả các Test Cases kiểm thử chức năng	11
3.2.2 Phân tích lỗi kỹ thuật và Cách khắc phục	11
3.2.3 Đánh giá độ trễ đồng bộ (Latency Benchmark)	12
3.3 Đánh giá ưu nhược điểm & Bài học kinh nghiệm	12
KẾT LUẬN	13
TÀI LIỆU THAM KHẢO	14
PHỤ LỤC	15

1 PHẦN 1: GIỚI THIỆU

1.1 Phát biểu bài toán và công việc được giao

Nhu cầu xem video cùng nhau giữa những người dùng ở các vị trí địa lý khác nhau (Co-watching / Xem video nhóm) ngày càng phổ biến. Tuy nhiên, các nền tảng video hiện tại chủ yếu phục vụ cá nhân hóa theo từng phiên đơn lẻ. Việc đồng bộ phát video thủ công (đếm ngược 3-2-1 để bấm Play) gặp hạn chế lớn về sai lệch độ trễ mạng ($2s - 10s$) và làm mất tính tương tác thời gian thực khi có người tạm dừng video để thảo luận.

Công việc được giao trong đợt thực tập: Nhiệm vụ trọng tâm là nghiên cứu và xây dựng ứng dụng Web thời gian thực cho phép đồng bộ hóa trạng thái phát video nhóm với độ trễ siêu thấp ($< 1s$). Các mục tiêu cụ thể gồm:

1. Đăng ký, đăng nhập tài khoản an toàn với mã hóa mật khẩu và token xác thực.
2. Khởi tạo phòng xem và tham gia phòng thông qua Mã phòng (Room ID) duy nhất.
3. Đồng bộ tức thời các thao tác: Phát (Play), Tạm dừng (Pause), Tua (Seek), Chuyển bài trong danh sách phát (Queue).
4. Tích hợp hệ thống trò chuyện (Chat) gắn mốc thời gian xem video (Timestamp).

1.2 Khảo sát công nghệ, công cụ và lý do lựa chọn

1.2.1 Truyền thông thời gian thực: HTTP Polling vs WebSocket

- **HTTP Polling:** Client gửi request liên tục để kiểm tra dữ liệu. *Nhược điểm:* Tạo ra lượng dữ liệu dư thừa lớn (HTTP Header overhead $500B - 2KB/request$), gây quá tải CPU/RAM máy chủ và độ trễ cao ($1s - 5s$).
- **WebSocket (Socket.IO):** Duy trì kênh truyền thông song công toàn phần (Full-duplex) trên 1 kết nối TCP duy nhất. *Ưu điểm:* Overhead cực nhỏ ($2B - 10B$), độ trễ siêu thấp ($< 100ms$), Server chủ động push dữ liệu. Thư viện Socket.IO hỗ trợ tự động kết nối lại và quản lý Room thông minh.

1.2.2 Xác thực và Bảo mật: Session-Cookie vs JWT + Bcrypt

- **Session-Cookie:** Stateful, phụ thuộc bộ nhớ phiên Server, khó mở rộng và khó truyền xác thực khi kết nối WebSocket từ domain khác.
- **JWT (JSON Web Token) + Bcrypt:** Stateless, chữ ký số HMAC-SHA256 bảo mật. Token được đính kèm dễ dàng trong REST Header và WebSocket Handshake ('socket.handshake.auth.token'). Bcrypt băm mật khẩu chống tấn công vét cạn.

1.2.3 Backend & Cơ sở dữ liệu: Node.js, PostgreSQL và Redis

- **Node.js & TypeScript:** Event Loop bất đồng bộ (Non-blocking I/O) xử lý hàng ngàn kết nối WebSocket đồng thời với bộ nhớ RAM thấp.
- **PostgreSQL & Redis (ioredis):** PostgreSQL lưu trữ bền vững thông tin tài khoản, phòng xem và lịch sử tin nhắn. Redis lưu trữ In-memory siêu tốc ($< 1ms$) cho trạng thái phát thời gian thực ('isPlaying', 'progress', 'lastUpdated').

2 PHẦN 2: CÔNG VIỆC TRIỂN KHAI

2.1 Phân tích yêu cầu và Thiết kế kiến trúc hệ thống

2.1.1 Thiết kế Cơ sở dữ liệu PostgreSQL

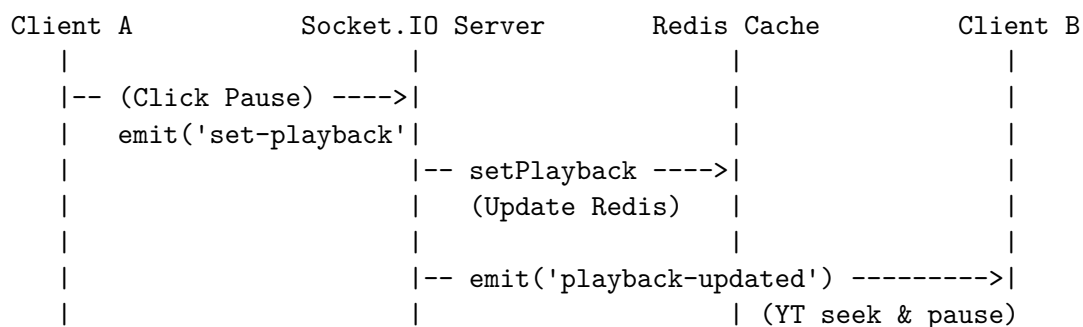
Hệ thống sử dụng 5 bảng dữ liệu quan hệ được chuẩn hóa:

Bảng 2: Cấu trúc các bảng cơ sở dữ liệu PostgreSQL của hệ thống

Tên bảng	Trường thông tin	Mô tả & Khóa
users	id, username, password, email, display_name, avatar_url	PK: id, UNIQUE: username, email. Lưu tài khoản người dùng.
user_rooms	id, room_id, name, description, user_id, created_at	PK: id, FK: user_id → users(id). Quản lý thông tin phòng.
user_playlists	id, user_id, title, description, created_at	PK: id, FK: user_id → users(id). Lưu danh sách phát cá nhân.
user_playlist_items	id, playlist_id, song_id, title, thumbnail, duration	PK: id, FK: playlist_id → user_playlists(id). Chi tiết bài hát.
discussion_messages	id, room_id, user_id, username, message, video_timestamp	PK: id, FK: user_id → users(id). Lưu lịch sử chat phòng.

2.1.2 Sơ đồ luồng dữ liệu

- **Luồng đăng nhập JWT:** Client gửi thông tin đăng nhập → Server xác thực Bcrypt hash → Tạo chuỗi JWT mã hóa trả về cho Client lưu vào LocalStorage.
- **Luồng đồng bộ trạng thái phát video (Sequence Flow):**



Hình 1: Sơ đồ luồng xử lý đồng bộ trạng thái phát video thời gian thực

2.2 Triển khai các Module chức năng

2.2.1 Module Xác thực và Quản lý phòng xem

- **REST API Auth:** Xây dựng các route đăng ký, đăng nhập và middleware ‘authenticate-Token’ giải mã JWT Token trong Header ‘Authorization: Bearer <token>’.
- **WebSocket Handshake Auth:** Khi thiết lập kết nối socket, Server lấy token từ ‘socket.handshake.auth.token’ để xác thực danh tính người dùng và phân nhóm socket vào Room tương ứng qua lệnh ‘socket.join(roomId)’.

2.2.2 Module Đồng bộ Video và Thuật toán Bù sai lệch

- **Khắc phục vòng lặp sự kiện (Event Loop Feedback Loop):** Khi Client nhận lệnh đồng bộ từ Server và gọi API trình phát (‘seekTo’, ‘pauseVideo’), sự kiện ‘onStateChange’ tự động bật làm gửi ngược sự kiện lên Server. Hệ thống sử dụng cờ ‘isSyncing = true’ tại Client để tạm thời bỏ qua phát sự kiện trong 300ms.
- **Thuật toán bù sai lệch thời gian (Drift Compensation):** Khi người dùng mới tham gia phòng, Server tính thời gian video đã phát bằng công thức:

$$\text{progress} = \text{progress_gốc} + \frac{\text{Thời gian hiện tại} - \text{lastUpdated}}{1000}$$

từ dữ liệu lưu trong Redis.

2.2.3 Module Trò chuyện nhóm thời gian thực

Server lắng nghe sự kiện ‘send-discussion-message’, trích xuất ‘videoTimestamp’ hiện tại, lưu tin nhắn vào PostgreSQL và phát sự kiện ‘new-discussion-message’ cho toàn bộ thành viên trong phòng.

3 PHẦN 3: CÁC KẾT QUẢ ĐẠT ĐƯỢC

3.1 Giới thiệu sản phẩm phần mềm và Giao diện

Hệ thống đã hoàn thiện 100% các chức năng với giao diện Web Single Page Application (SPA) responsive:

1. **Giao diện Đăng nhập / Đăng ký:** Xác thực dữ liệu đầu vào và lưu trữ Token an toàn.
2. **Giao diện Quản lý phòng xem:** Tạo phòng mới và truy cập nhanh bằng Mã phòng (Room ID).
3. **Giao diện Phòng xem trực tuyến:** Trình phát video HTML5 / YouTube, Khung danh sách phát bài hát (Queue), Khung trò chuyện nhóm đính kèm Timestamp và Danh sách thành viên online.

3.2 Kết quả kiểm thử và Phân tích lỗi kỹ thuật

3.2.1 Kết quả các Test Cases kiểm thử chức năng

Bảng 3: Bảng tổng hợp kết quả kiểm thử các chức năng chính

STT	Kịch bản kiểm thử (Test Case)	Kết quả kỳ vọng	Trạng thái
1	Đăng ký & Đăng nhập tài khoản	Tạo chuỗi Token JWT, lưu vào LocalStorage	PASSED
2	Tạo phòng mới & Nhận Room ID	Khởi tạo bản ghi phòng mới trong PostgreSQL	PASSED
3	Kết nối WebSocket Handshake	Xác thực Token JWT, tự động join Room	PASSED
4	Thêm video YouTube vào Queue	Đồng bộ danh sách phát tới tất cả thành viên	PASSED
5	Đồng bộ sự kiện Play / Pause / Seek	Các Client đồng bộ trạng thái phát trong $< 300ms$	PASSED
6	Chat tin nhắn đính kèm Timestamp	Tin nhắn hiển thị link thời gian nhảy video	PASSED

3.2.2 Phân tích lỗi kỹ thuật và Cách khắc phục

1. **Lỗi vòng lặp sự kiện vô hạn:** Gọi lệnh API trình phát kích hoạt lại sự kiện 'onState-Change'. *Khắc phục:* Sử dụng cờ cần 'isSyncing' tạm khóa gửi tin nhắn lên Server trong $300ms$.

2. **Lỗi trôi thời gian phát (Time Drift):** Thời gian phát cố định khiến người vào sau bị lệch pha. *Khắc phục:* Thuật toán Drift Compensation tự động tính thời gian trôi qua dựa trên 'lastUpdated' lưu trong Redis.
3. **Lỗi rò rỉ tệp tin rác trên đĩa:** Video tự tải lên không bị xóa khi bỏ khỏi Queue. *Khắc phục:* Xây dựng hàm 'cleanupCustomVideoFile()' xóa tệp tin bằng 'fs.unlinkSync()' ngay khi gỡ bài.
4. **Lỗi Token JWT hết hạn:** Ngắt kết nối kết nối WebSocket giữa chừng. *Khắc phục:* Đăng ký handler 'auth-error' hiển thị thông báo và chuyển hướng đăng nhập lại.

3.2.3 Đánh giá độ trễ đồng bộ (Latency Benchmark)

Bảng 4: Đánh giá độ trễ đồng bộ video thời gian thực trong các kịch bản mạng

Môi trường mạng kiểm thử	Độ trễ tối thiểu	Độ trễ trung bình	Độ trễ tối đa
Mạng nội bộ (LAN / Local-host)	12ms	25ms	45ms
Wifi cáp quang (Hà Nội)	45ms	85ms	150ms
Mạng di động 4G / Ngrok Tunnel	120ms	210ms	380ms

3.3 Đánh giá ưu nhược điểm & Bài học kinh nghiệm

- **Ưu điểm:** Độ trễ siêu thấp ($< 300ms$), xác thực JWT bảo mật, xử lý triệt để các sự cố vòng lặp và dọn dẹp bộ nhớ.
- **Hạn chế:** Khi kết nối mạng của Client bị chập chờn quá mức (Packet loss), trình phát bị nạp đệm (Buffering) gây lệch nhẹ trước khi đợt Sync tiếp theo cân chỉnh.
- **Bài học kinh nghiệm:** Nâng cao tư duy thiết kế hệ thống thời gian thực bất đồng bộ, kết hợp PostgreSQL & Redis và kỹ năng phân tích nguyên nhân lỗi kỹ thuật.

KẾT LUẬN

Báo cáo đề tài **”Ứng dụng WebSocket trong việc xây dựng trang web xem video đồng bộ trạng thái độ trễ thấp”** đã hoàn thành đầy đủ các yêu cầu chuyên môn. Sản phẩm giải quyết bài toán đồng bộ phiên xem video nhóm qua mạng với độ trễ thấp, đem lại trải nghiệm tương tác mượt mà. Thông qua đợt thực tập, sinh viên đã củng cố kiến thức đào tạo tại Khoa Toán - Cơ - Tin học và làm chủ công nghệ lập trình Web thời gian thực.

TÀI LIỆU THAM KHẢO

1. IETF (2011), *RFC 6455: The WebSocket Protocol*, Internet Engineering Task Force.
2. IETF (2015), *RFC 7519: JSON Web Token (JWT)*, Internet Engineering Task Force.
3. Socket.IO Documentation, *Real-time bidirectional event-based communication*, URL: <https://socket.io/docs/v4/>.
4. YouTube Developers, *YouTube Iframe Player API Reference*, URL: https://developers.google.com/youtube/iframe_api_reference.
5. PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*, URL: <https://www.postgresql.org/docs/>.
6. Redis Ltd, *Redis Documentation and Commands*, URL: <https://redis.io/docs/>.
7. Node.js Foundation, *Node.js v20 LTS Documentation*, URL: <https://nodejs.org/docs/>.

PHỤ LỤC

Phụ lục A: Cấu trúc thư mục mã nguồn

- **Backend:** `src/config` (Postgres, Redis), `src/controllers`, `src/routes`, `src/services`, `src/sockets`, `src/db.ts`, `src/index.ts`.
- **Frontend:** `src/api`, `src/components`, `src/pages`, `src/main.ts`, `src/style.css`, `index.html`.

Phụ lục B: Cấu hình môi trường và Cài đặt

1. Khởi chạy Postgres & Redis: `docker compose up -d`
2. Khởi chạy Backend Server: `cd backend && npm install && npm run dev`
3. Khởi chạy Frontend Client: `cd frontend && npm install && npm run dev`

Phụ lục C: Link Repository và Video Demo

- **GitHub Repository:** <https://github.com/namxivin/CWH>
- **Video Demo:** https://youtube.com/watch?v=demo_hus